

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

GPUs (Part 2)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Kayvon Fatahalian, Justin Hensley, Wen-mei Hwu, Natalie Enright Jerger, David Kirk, Mikko Lipasti and David Luebke

Last Lecture

- Buffer backpressure
- Router microarchitecture
- GPUs

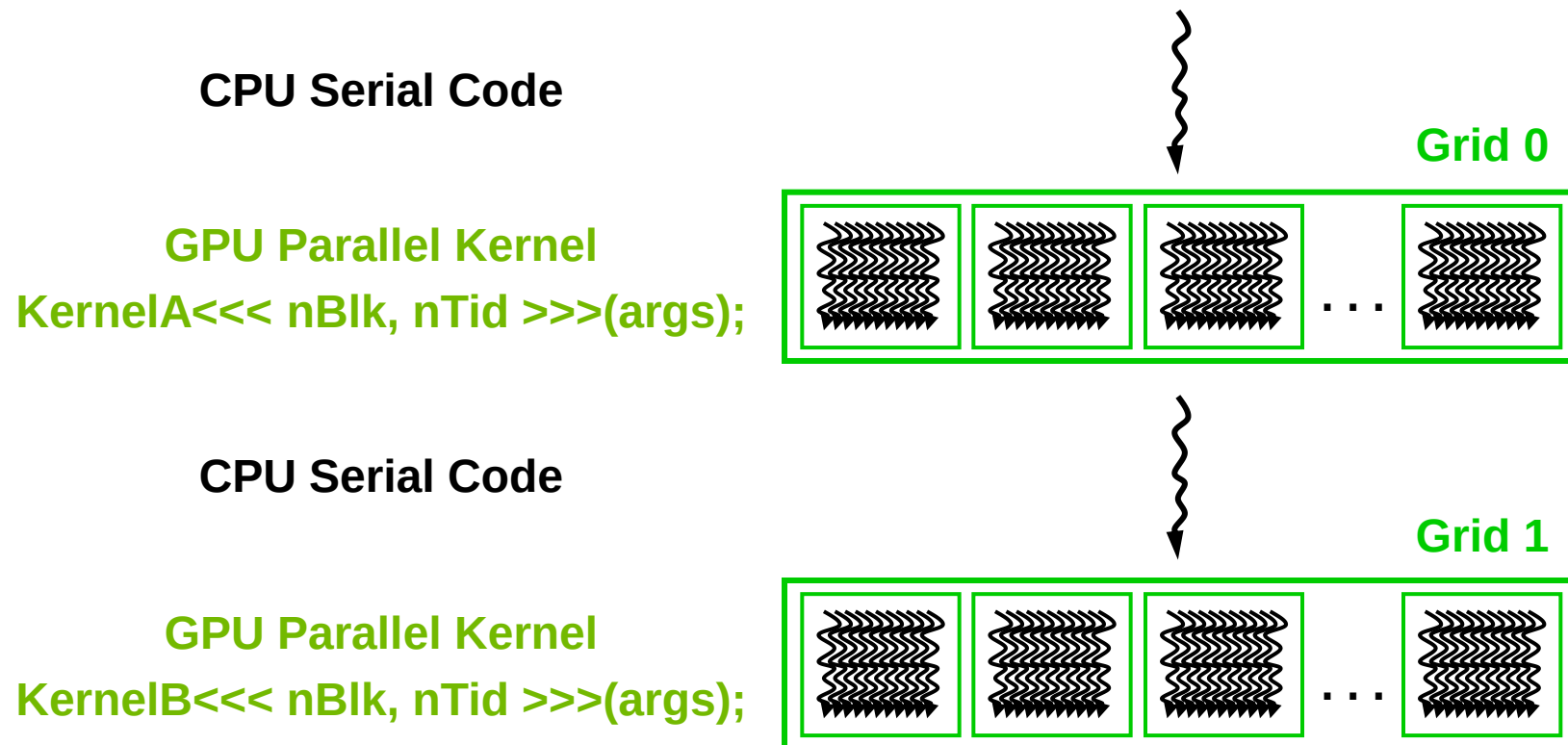
This Lecture

- GPU architecture
- GPU memory
- GPU coherence
- CUDA programming

GPU Threading

CUDA integrated CPU+GPU program

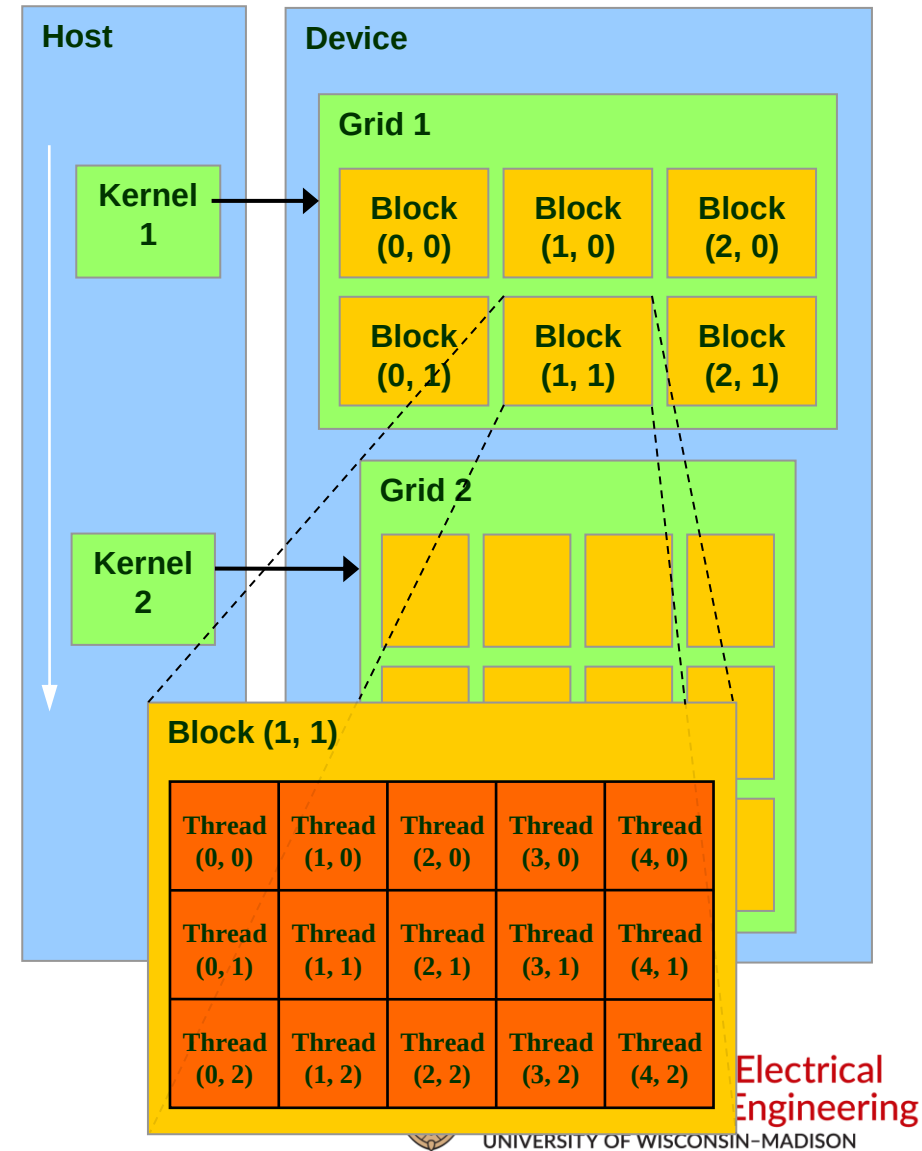
- Serial C code executes on CPU
- Parallel kernel C code executes on GPU thread blocks



GPU Threading

A kernel is executed as a *grid of thread blocks*

- All threads share memory space



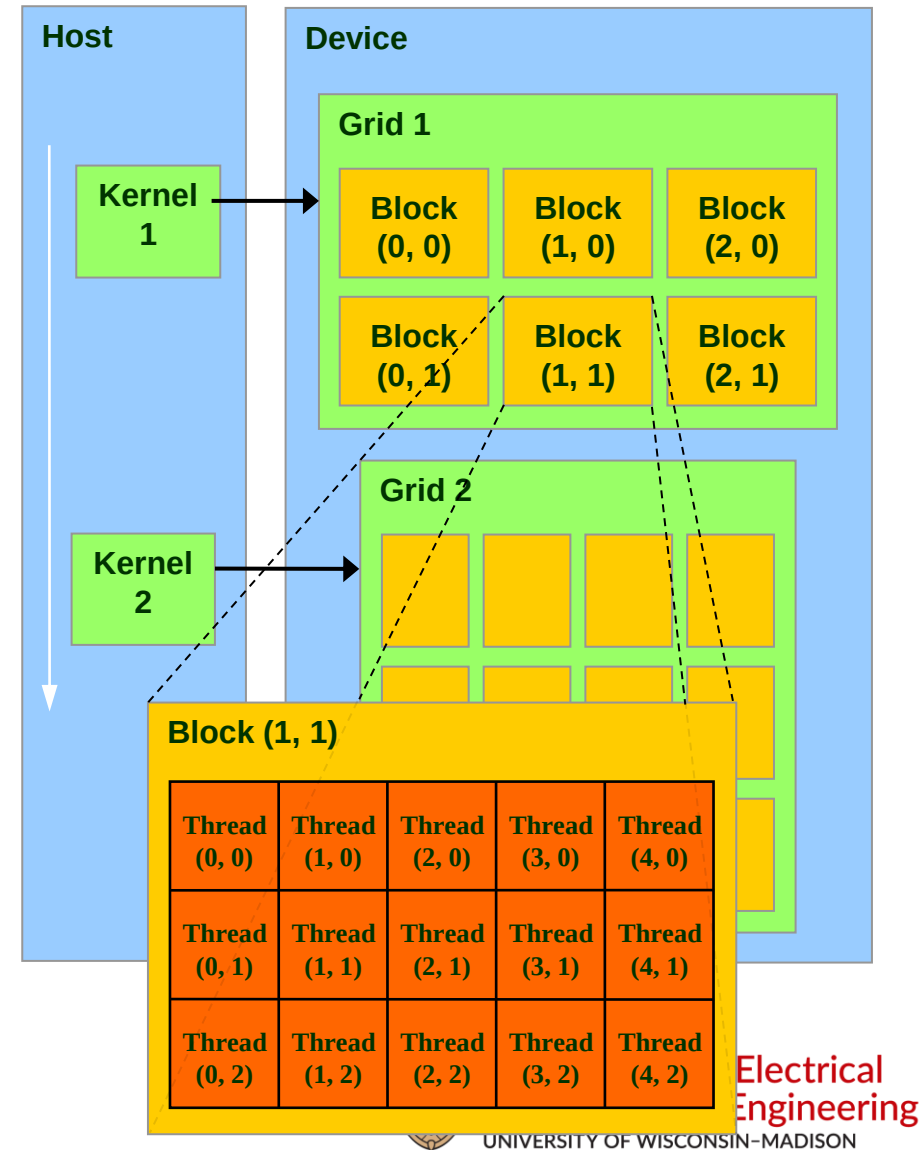
GPU Threading

A kernel is executed as a *grid of thread blocks*

- All threads share memory space

A thread block is a batch of threads that can cooperate with each other:

- Synchronizing their execution, for hazard-free shared memory accesses
- Data transfer through a low-latency shared memory
- Two threads from two different blocks cannot cooperate

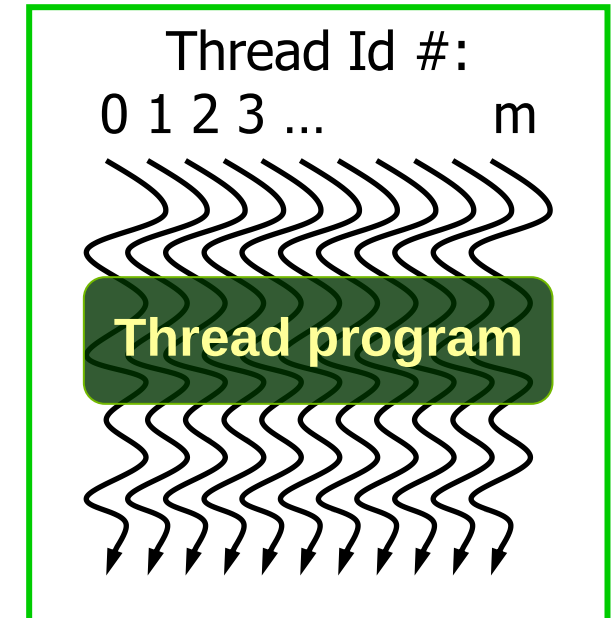


GPU Threading

Thread blocks:

- All threads in a thread block execute on same machine
- Programmer declares block size (e.g., 1-100s of concurrent threads), block shape (e.g., 1D, 2D, 3D), block dimensions (in threads)
- Each thread has thread ID within block; uses this thread ID to select which work to do and which data addresses to access

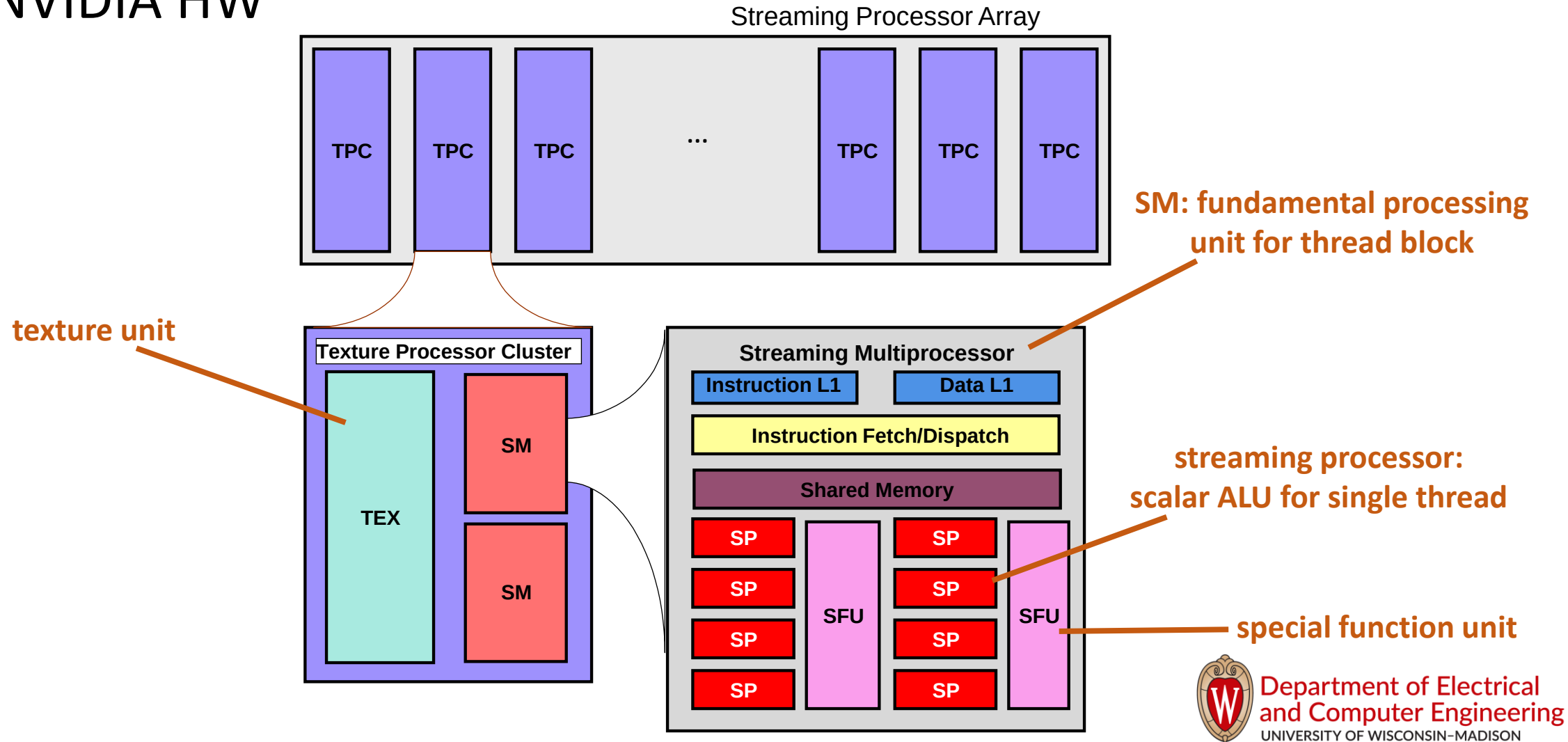
CUDA Thread Block



source: John Nickolls, NVIDIA

GPU Architecture

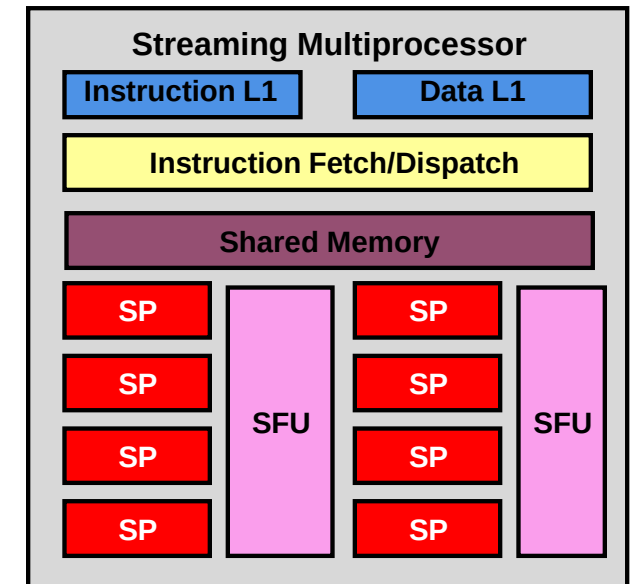
e.g., NVIDIA HW



GPU Architecture

Streaming Multiprocessor (SM):

- Multithreaded instruction dispatch
- Supports many (100s-1000s) active thread contexts
- Shared instruction fetch per chunk (*warp*) of threads
- e.g., Typically:
 - 32 threads per warp
 - 16-128 KB shared memory



GPU Architecture – Thread Blocks

Streaming processor array launches grid

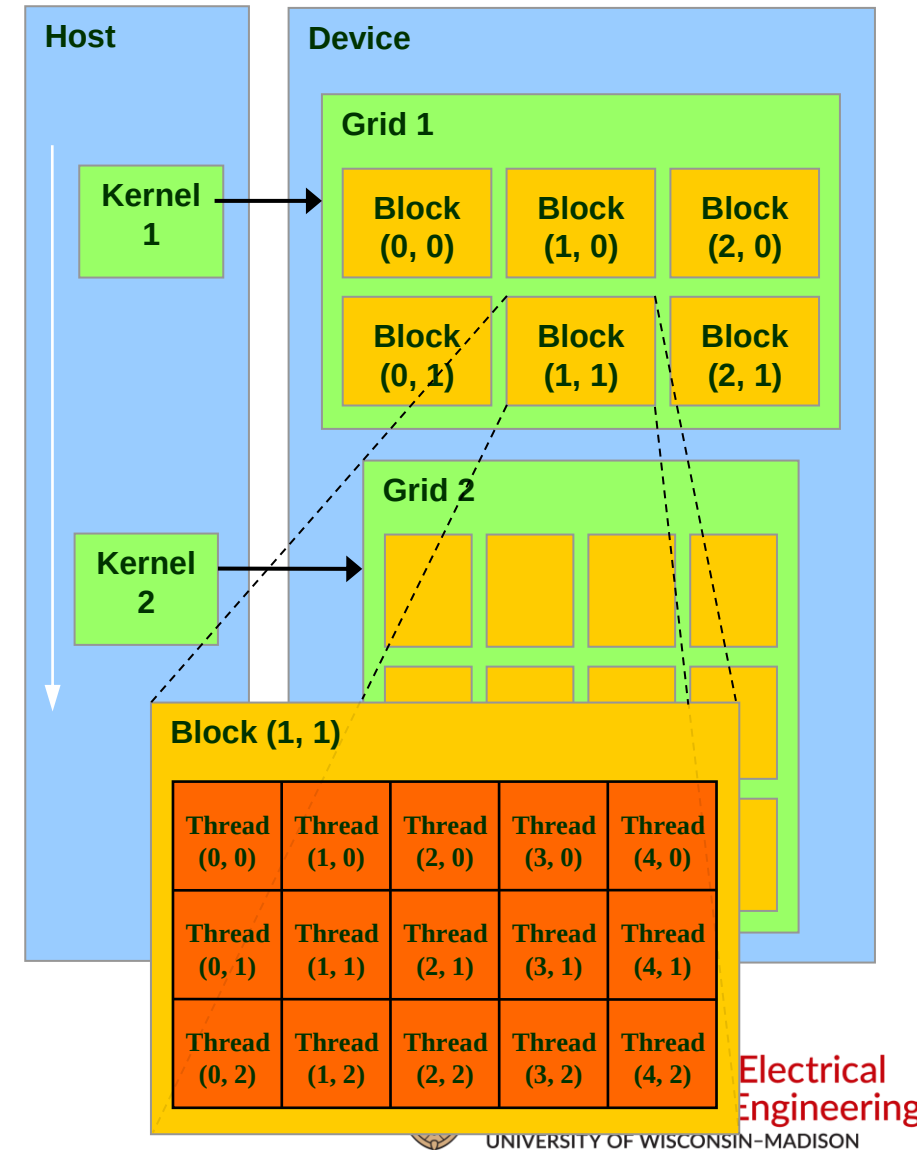
- Thread blocks are serially distributed to all SMs

Each SM launches warps of threads

- SM schedules and executes (time-multiplexes) warps that are ready to run

As warps and thread blocks complete, resources are freed

- Streaming processor array can distribute more thread blocks



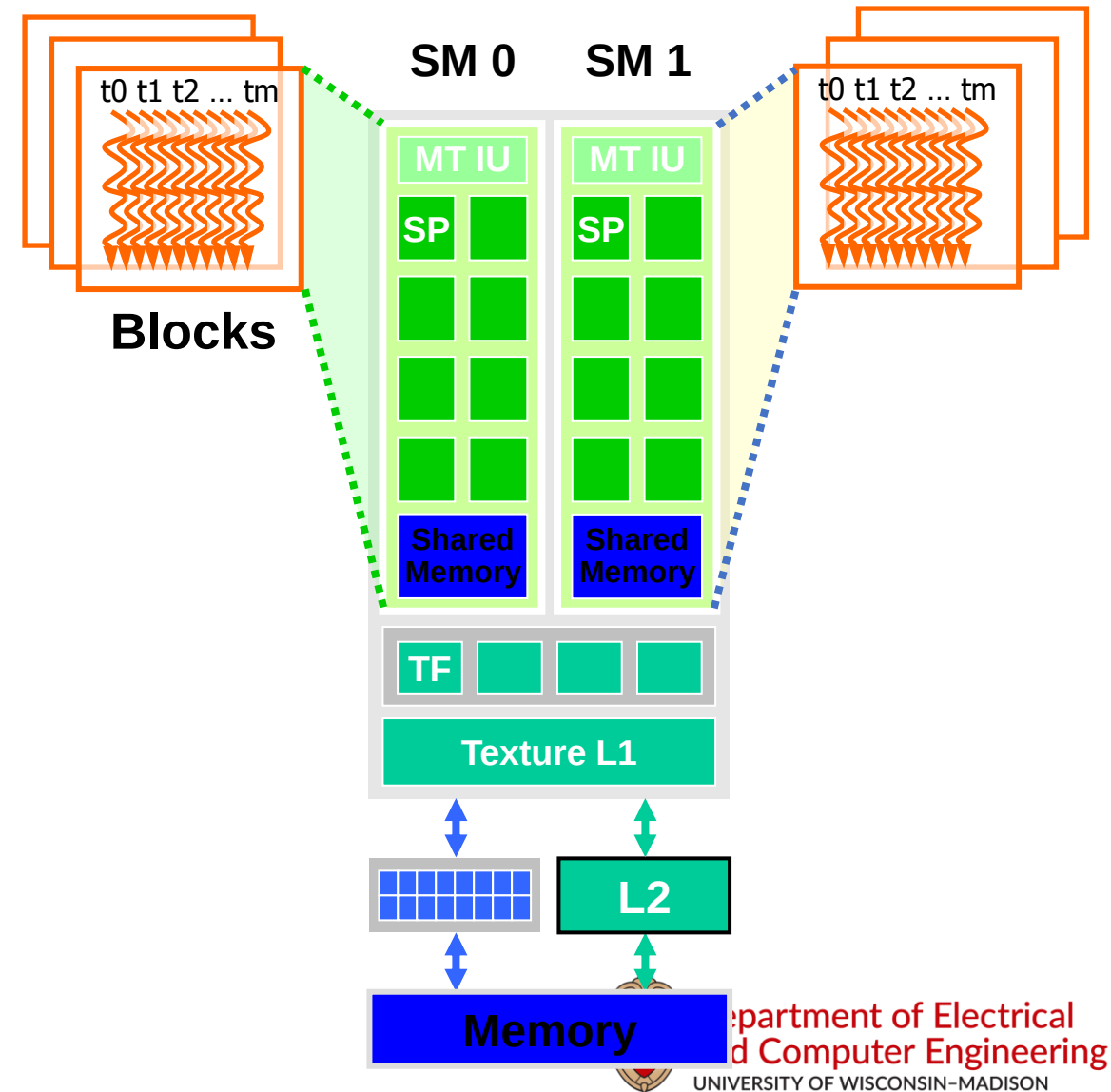
GPU Architecture – Thread Blocks

Threads are assigned to SMs in block granularity

- 10s of blocks per SM as resources allow
- 100s-1000s of threads per SM

Threads run concurrently

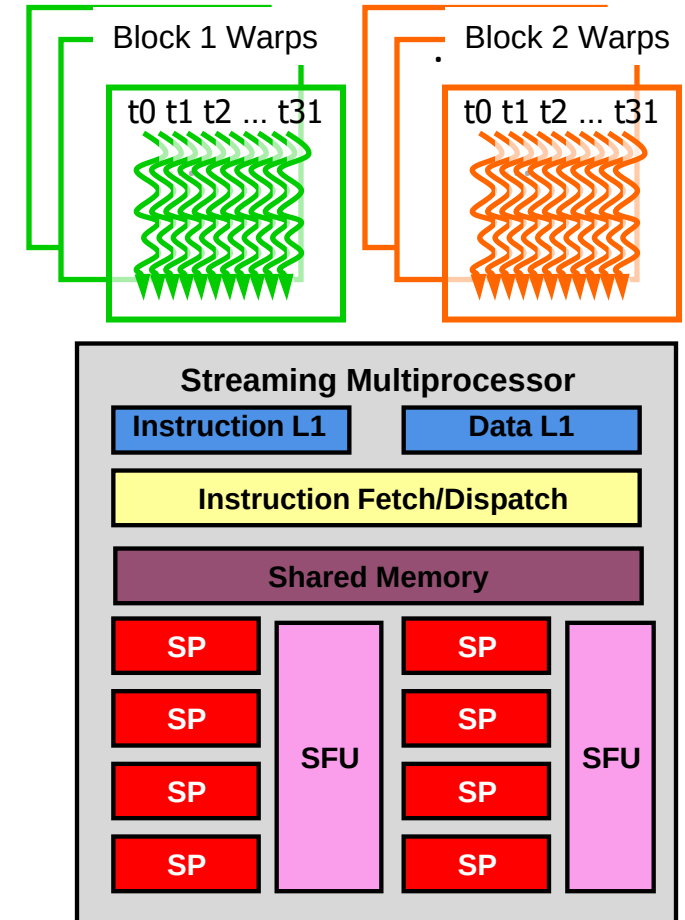
- SM assigns and maintains thread IDs
- SM manages and schedules thread execution



GPU Architecture – Warps

Each thread block is divided into *warps*

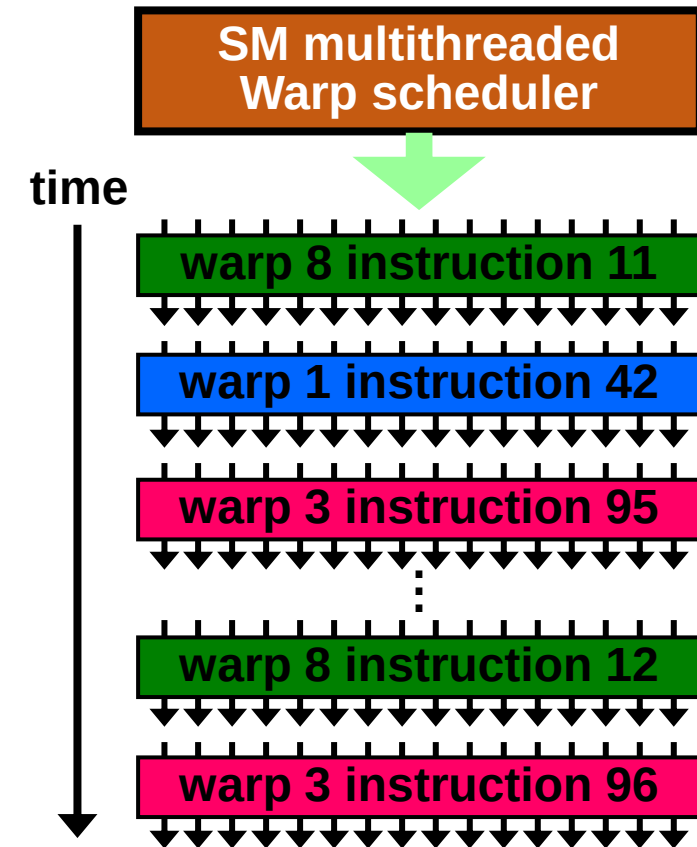
- HW implementation decision (not defined by programming model)
- Warp is a set of threads that always execute in lockstep
- Warps are the units for scheduling inside SM
 - At any point in time, only one warp is selected for instruction fetch



GPU Architecture – Warps

SM hardware implements zero-overhead warp scheduling

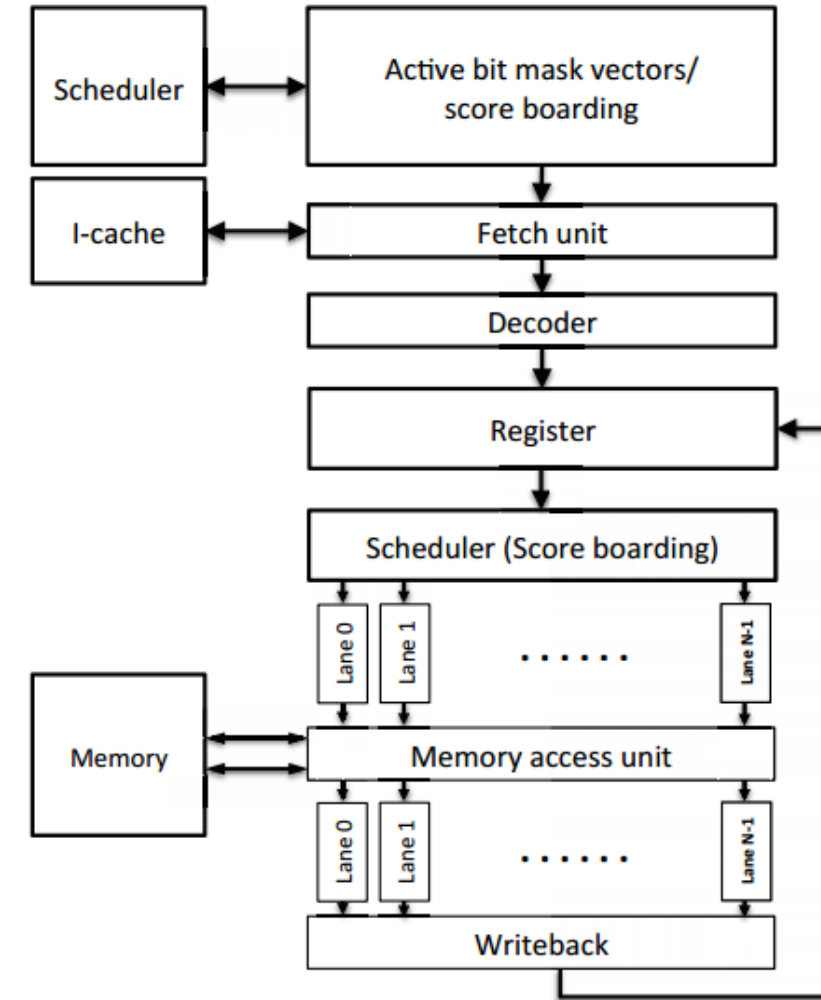
- Warps whose next instruction has its operands ready are eligible for execution
- Eligible warps are selected for execution via prioritized scheduling policy
- Interleave warp execution to hide latencies (e.g., memory)



GPU Architecture – Warps

Fetch one *warp instruction* per cycle

- From instruction L1 cache
- Into any instruction buffer slot



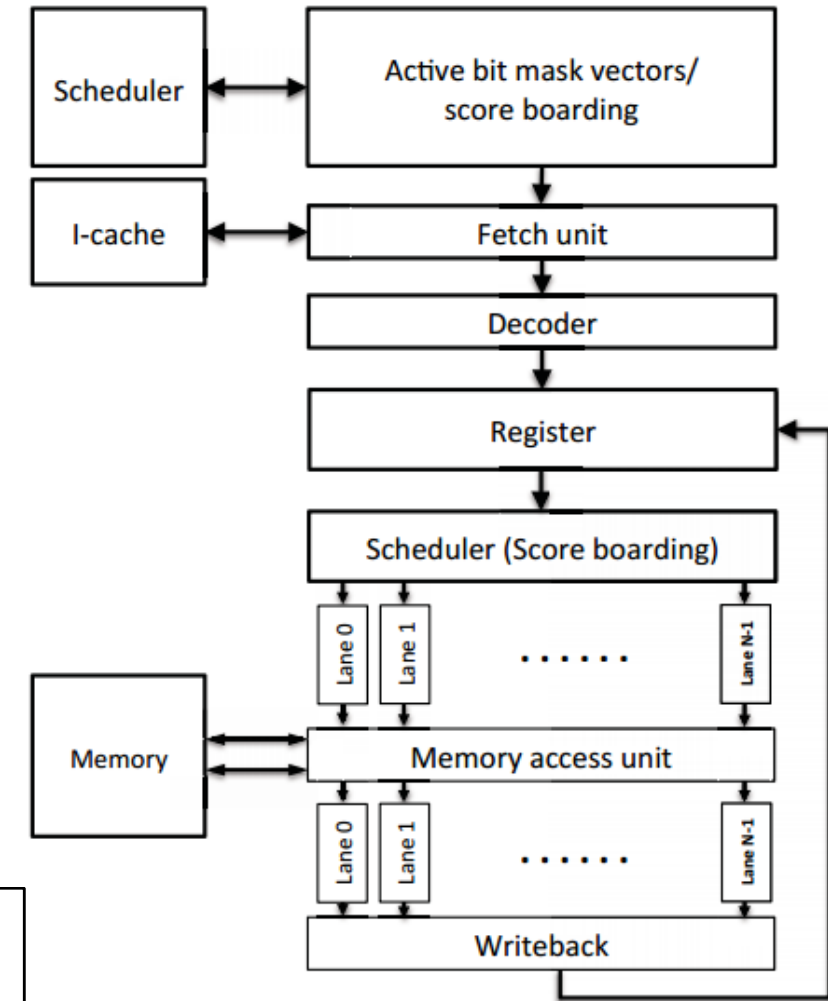
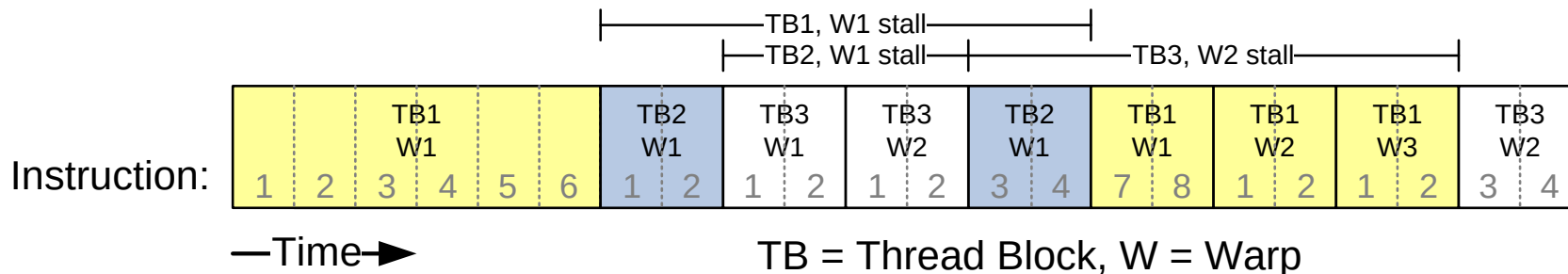
GPU Architecture – Warps

Fetch one *warp instruction* per cycle

- From instruction L1 cache
- Into any instruction buffer slot

Issue one ready-to-go warp instruction per cycle

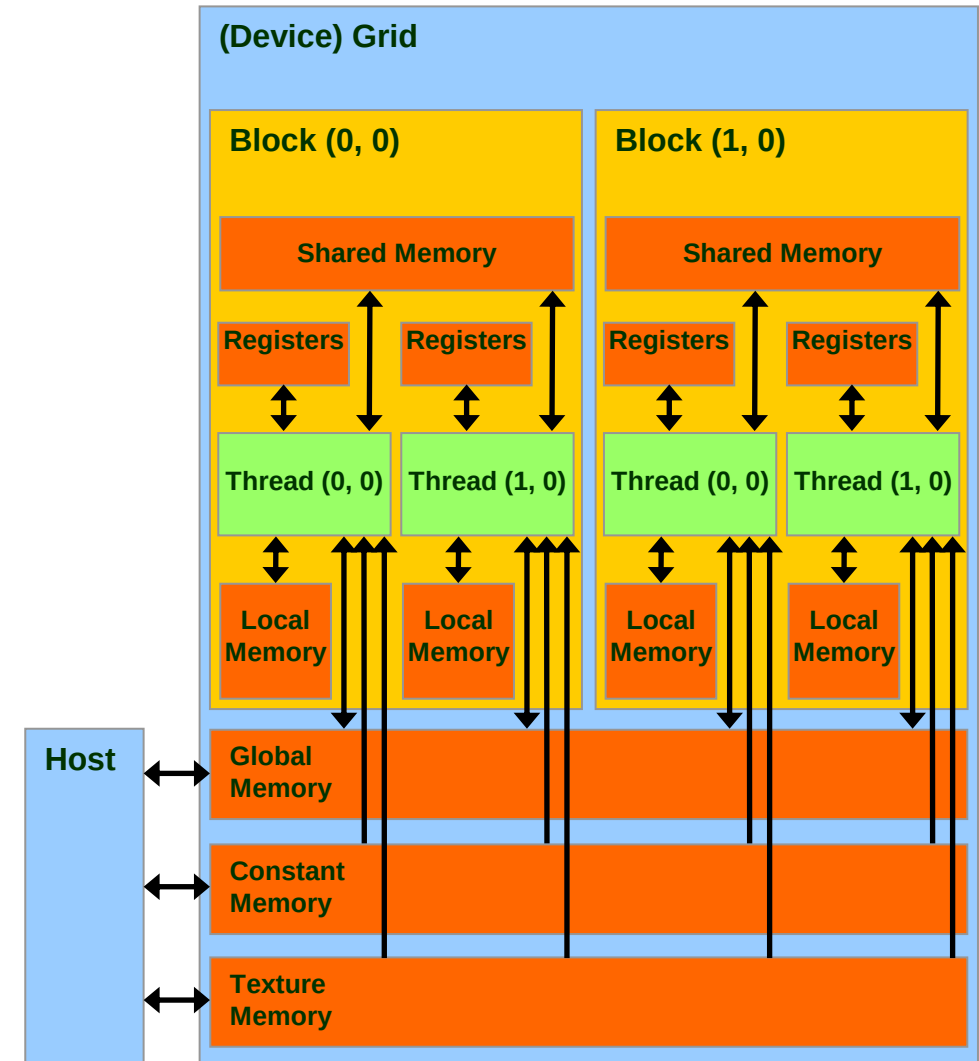
- From any warp
- Issued to execution lanes
- Operand scoreboarding used to prevent hazards
- Issue selection based on round-robin/age of warp



GPU Memory

Each GPU thread can:

- R/W per-thread *registers*
- R/W per-thread *local memory*
- R/W per-block *shared memory*
- R/W per-grid *global memory*
- R only per-grid *constant memory*
- R only per-grid *texture memory*



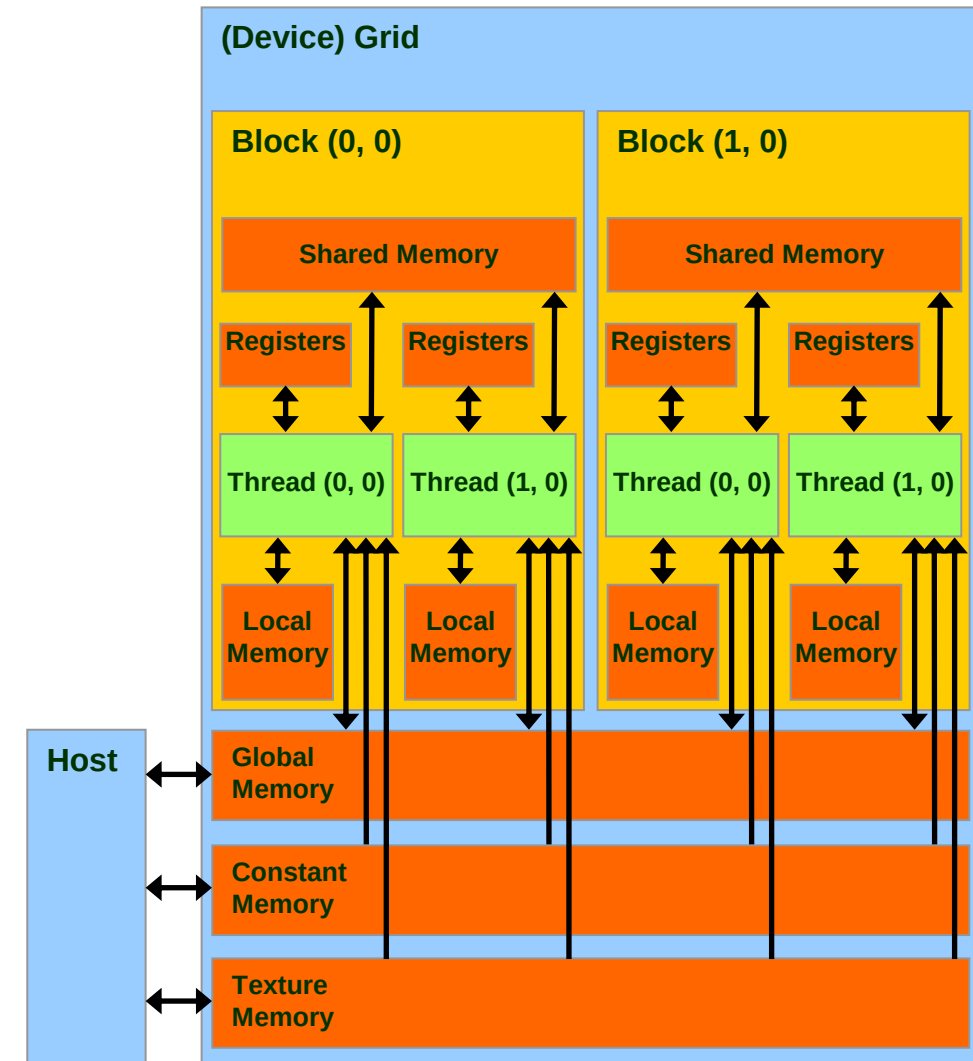
GPU Memory

Each GPU thread can:

- R/W per-thread *registers*
- R/W per-thread *local memory*
- R/W per-block *shared memory*
- R/W per-grid *global memory*
- R only per-grid *constant memory*
- R only per-grid *texture memory*

Host (CPU) can:

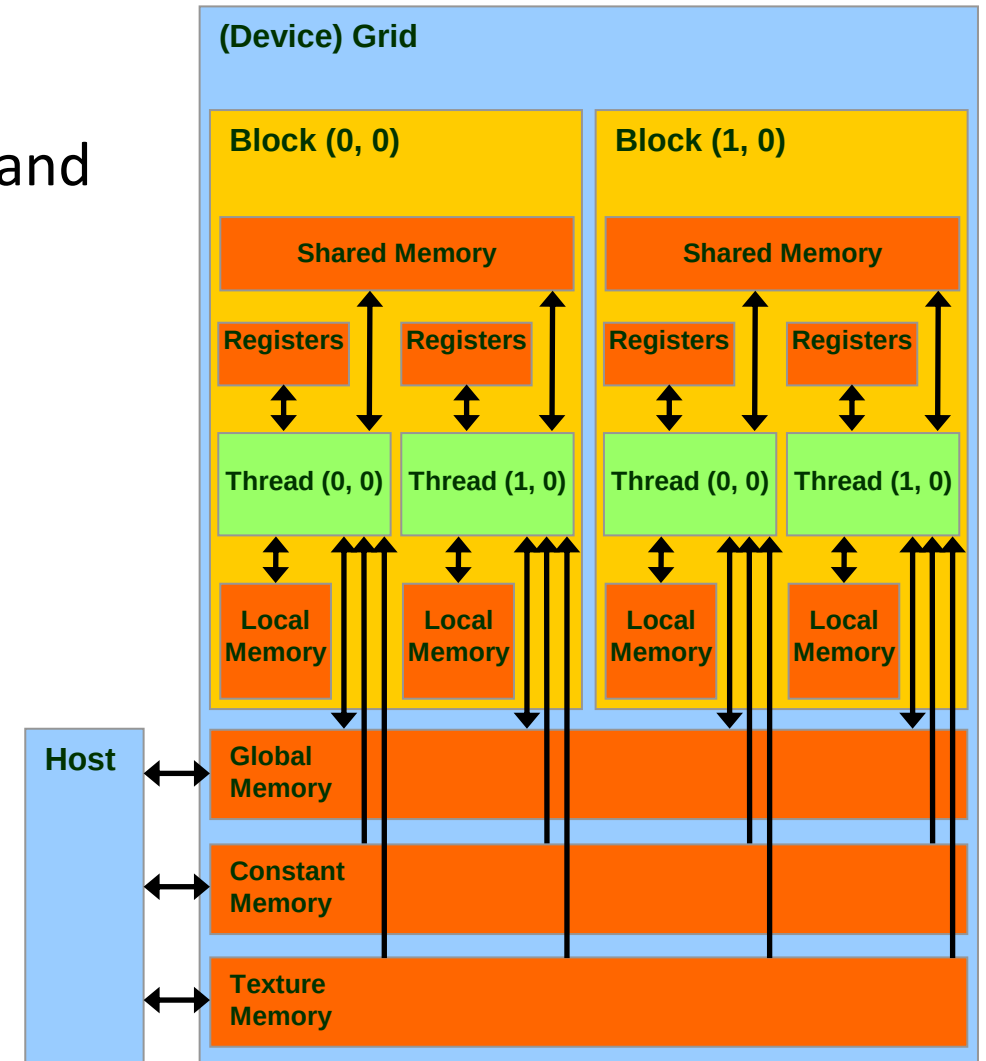
- R/W global memory
- R/W constant memory
- R/W texture memory



GPU Memory

Global memory:

- Main means of communicating data between host and GPU
- Contents visible to all threads



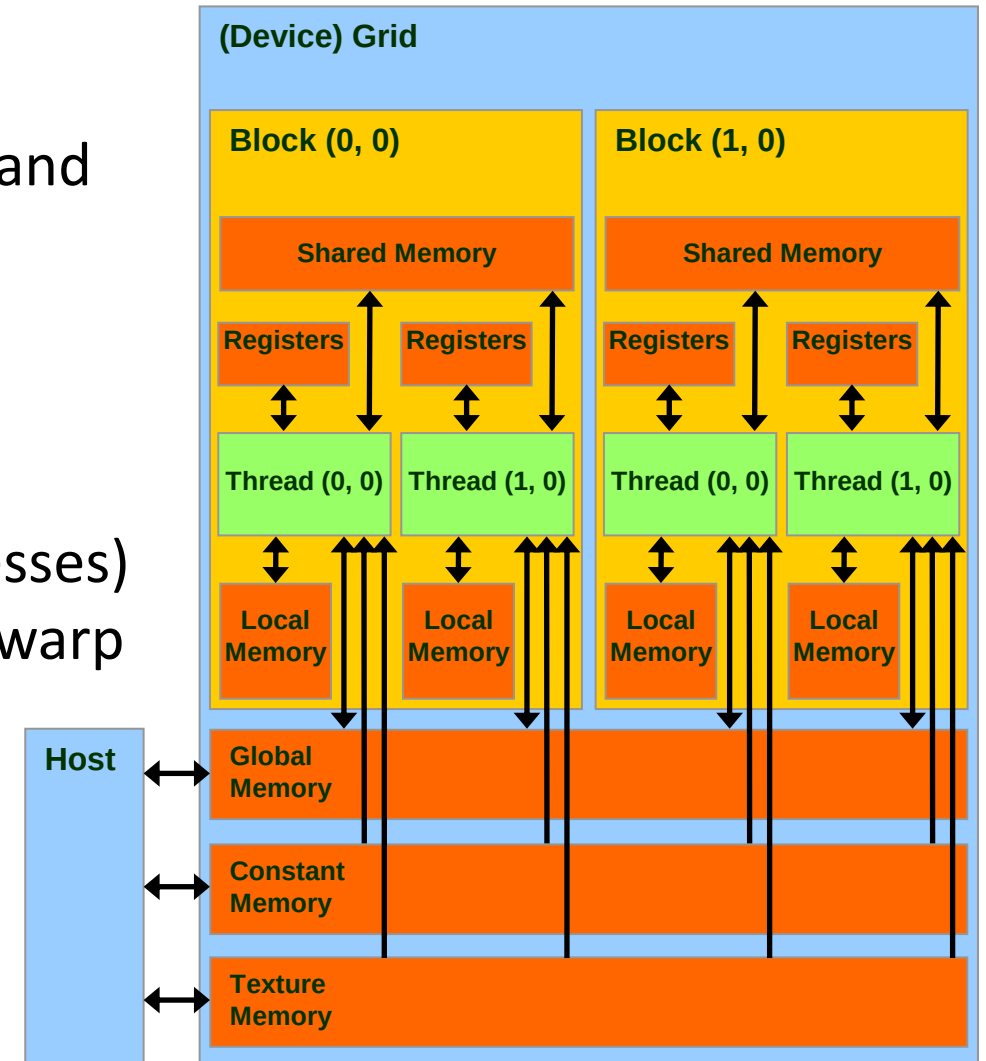
GPU Memory

Global memory:

- Main means of communicating data between host and GPU
- Contents visible to all threads

Constant memory:

- Constants initialized by host (e.g., immediate addresses)
- A constant value can be broadcast to all threads in warp
- Cached on-chip in L1 per SM



GPU Memory

Global memory:

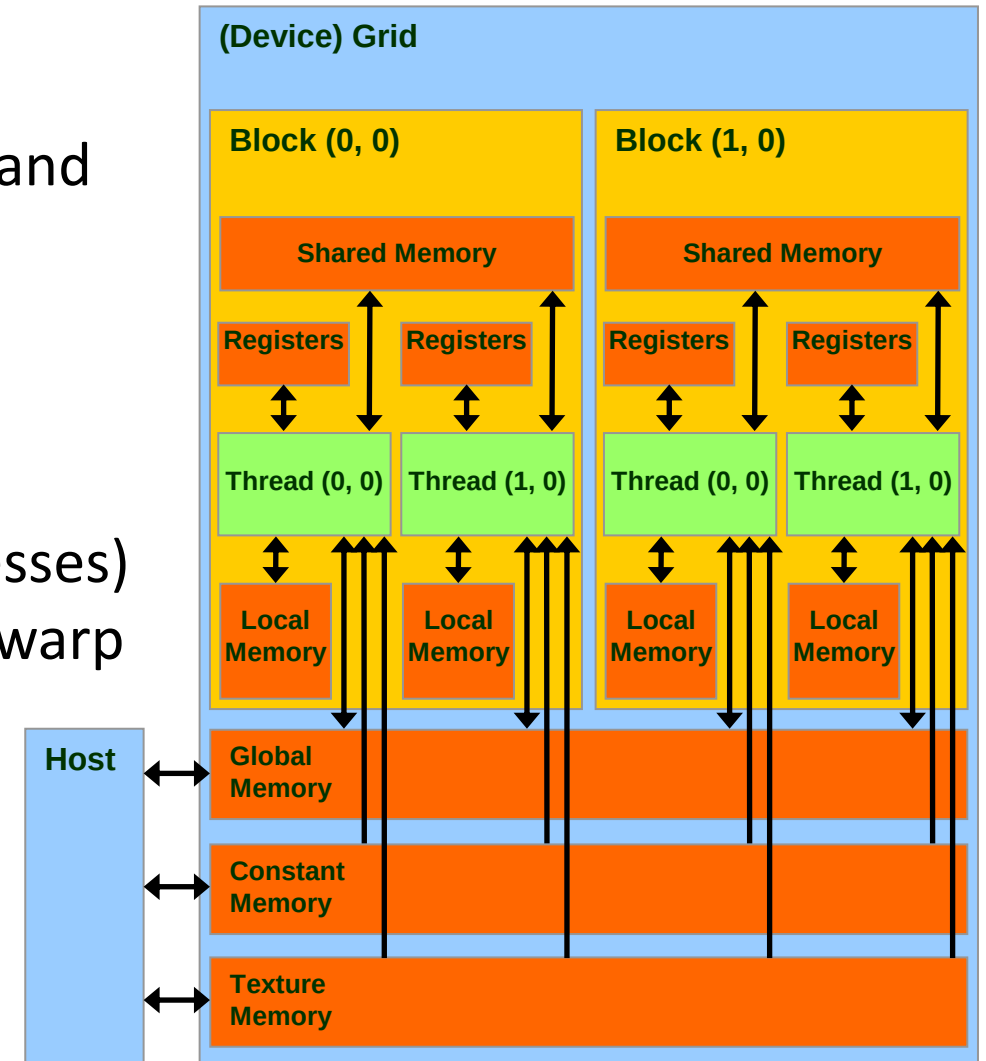
- Main means of communicating data between host and GPU
- Contents visible to all threads

Constant memory:

- Constants initialized by host (e.g., immediate addresses)
- A constant value can be broadcast to all threads in warp
- Cached on-chip in L1 per SM

Texture memory:

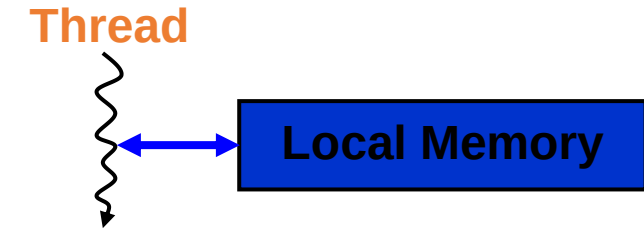
- Texture maps, cached on-chip in texture caches
- HW support for FP conversion and interpolation



GPU Memory

Local memory:

- Private per thread
- Auto variables, register spill



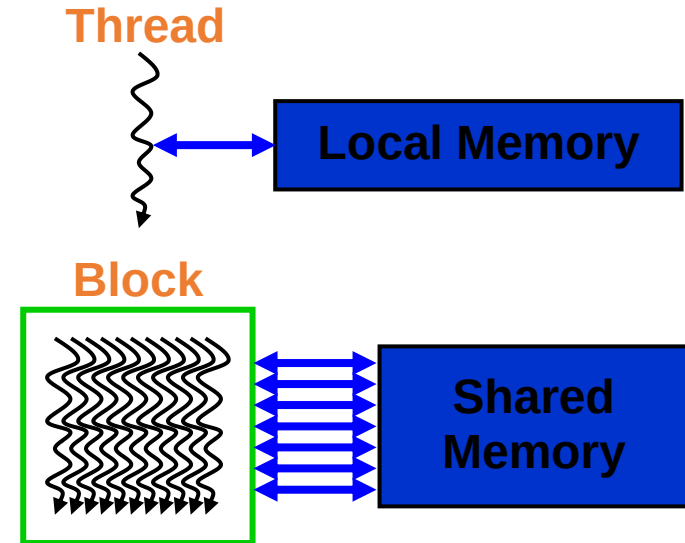
GPU Memory

Local memory:

- Private per thread
- Auto variables, register spill

Shared memory:

- Shared by threads per block
- Inter-thread communication



GPU Memory

Local memory:

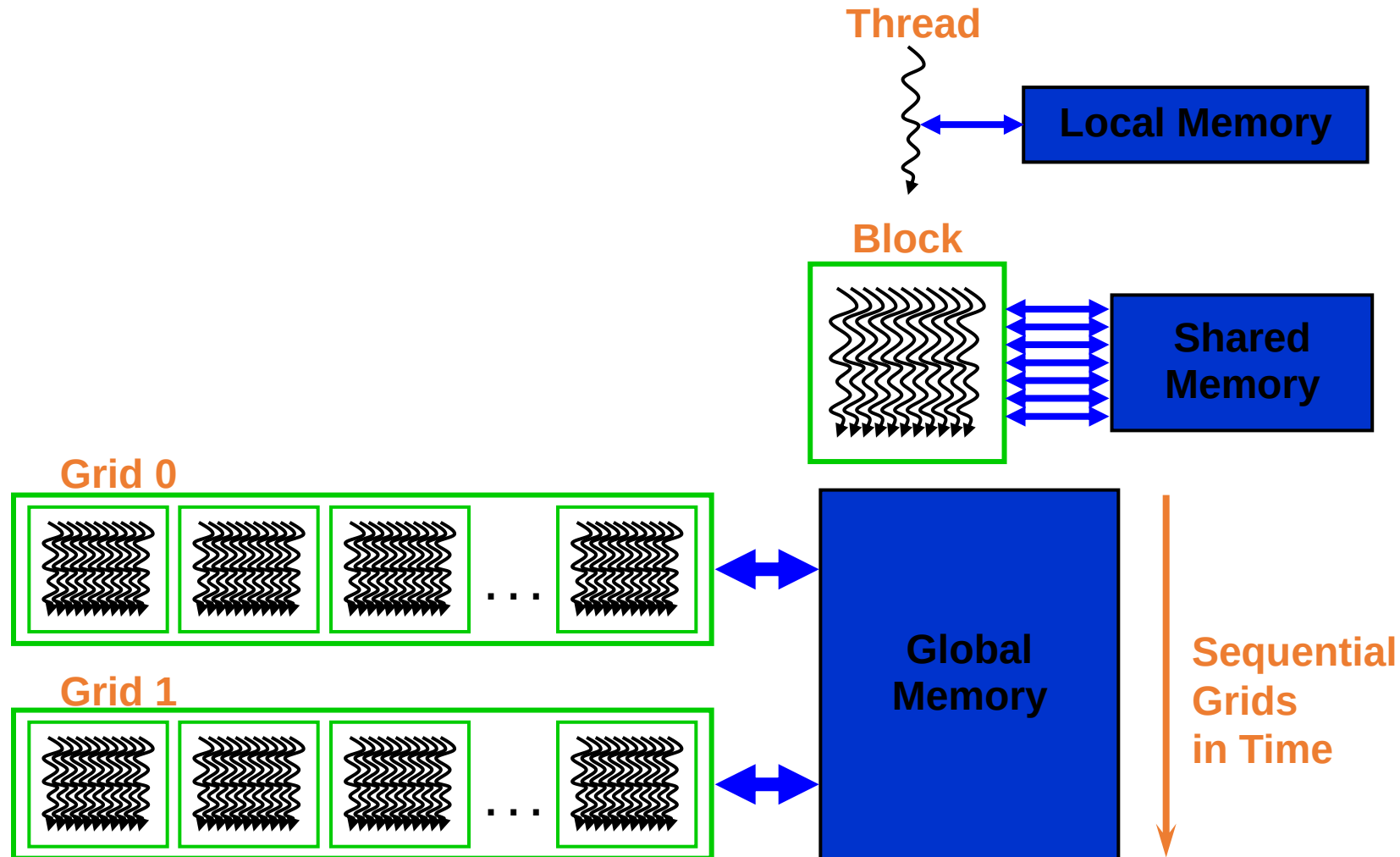
- Private per thread
- Auto variables, register spill

Shared memory:

- Shared by threads per block
- Inter-thread communication

Global memory:

- Shared by all threads
- Inter-grid (i.e., inter-kernel) communication per application



GPU Memory

Register file:

- 10-100s KBs of physical registers per SM (not architectural registers)
- Registers are dynamically partitioned across all blocks assigned to SM
- Once assigned to a block, a register is not accessible by threads in other blocks
- Banked SRAM, provides multiple operands per cycle

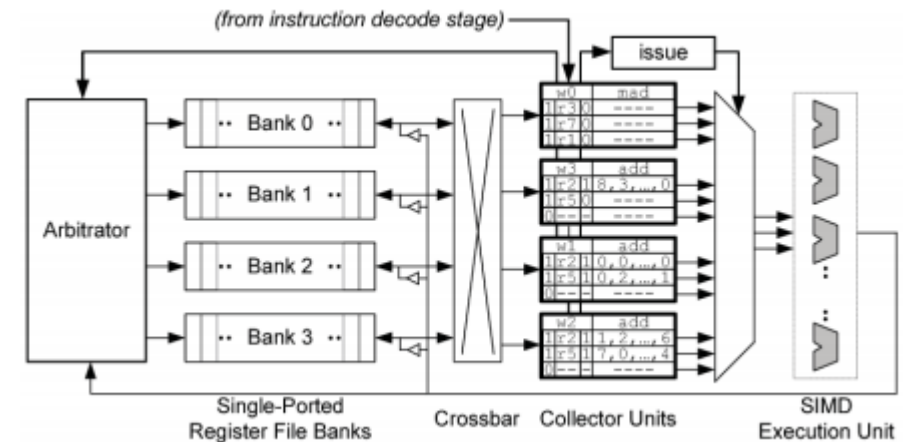
GPU Memory

Register file:

- 10-100s KBs of physical registers per SM (not architectural registers)
- Registers are dynamically partitioned across all blocks assigned to SM
- Once assigned to a block, a register is not accessible by threads in other blocks
- Banked SRAM, provides multiple operands per cycle

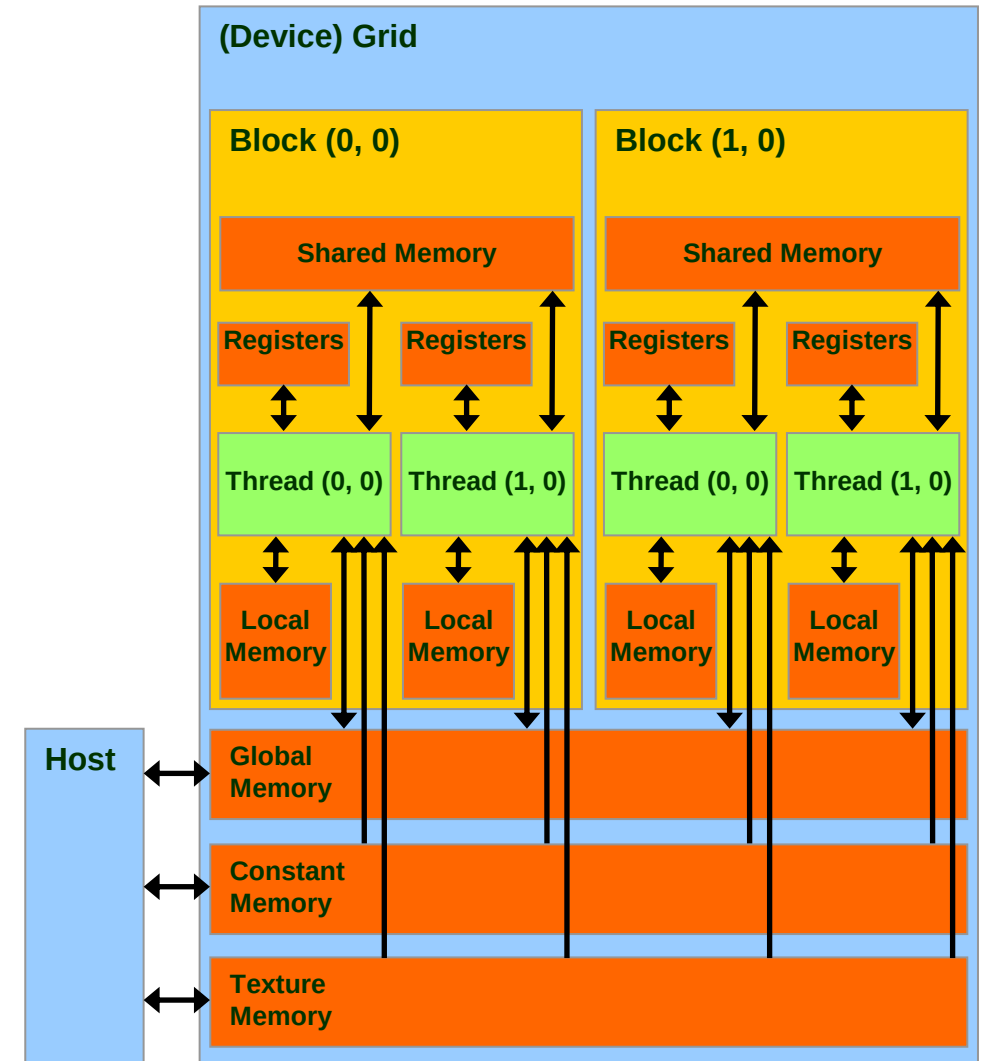
Operand collector:

- Requests register file reads, buffers values
- Resolves bank conflicts to register file banks



GPU Memory

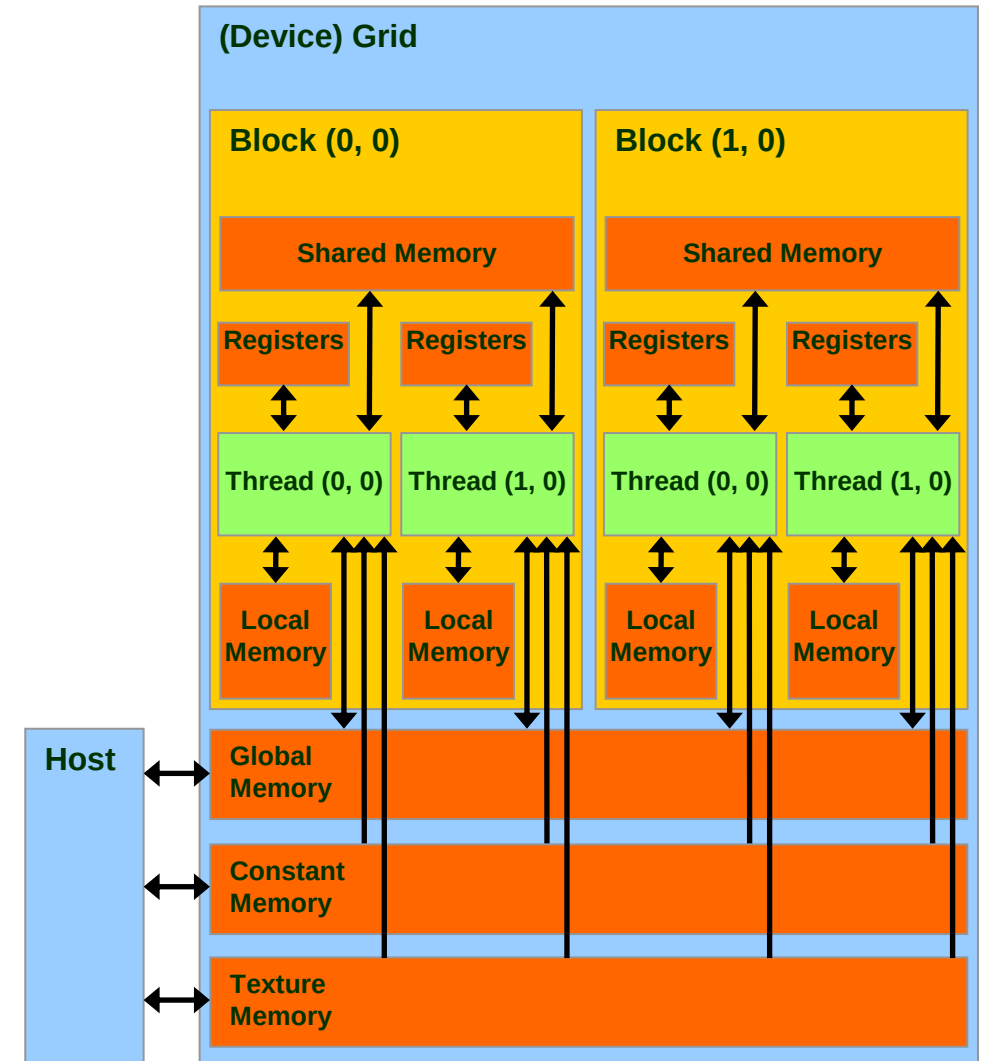
How is this memory hierarchy different from that of a CMP memory hierarchy?



GPU Memory

How is this memory hierarchy different from that of a CMP memory hierarchy?

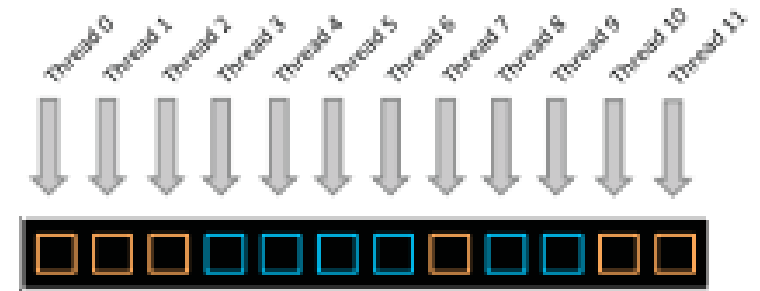
Explicit, the programmer defines where data resides in the hierarchy



GPU Memory – Memory Coalescing

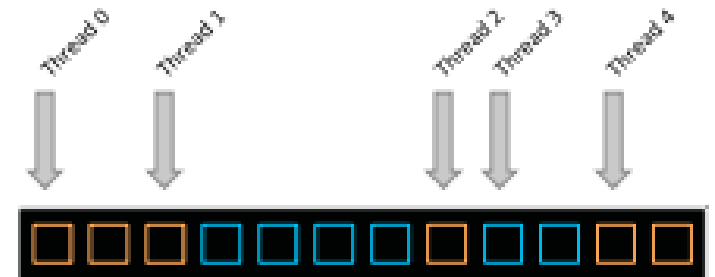
Within warp, if thread accesses are to adjacent locations (same memory block):

- *Coalescing* logic merges them into a single memory request
- Yields fewer memory requests



Within warp, if thread accesses are not coalesced:

- Memory requests are serialized
- Higher latency, less throughput, more energy

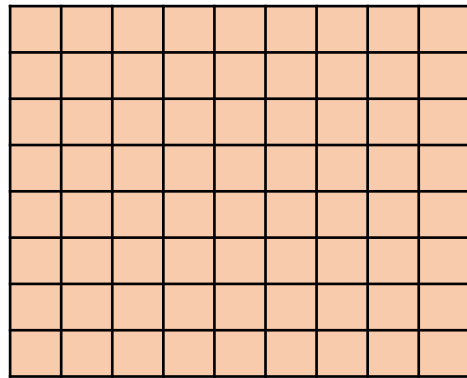


GPU Memory – Memory Coalescing

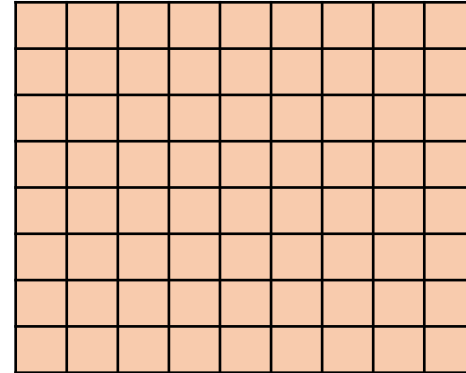
Within warp, we want threads to access the same memory block

- e.g., If we had the choice of parallelizing an array along rows vs. columns:

CPU



GPU

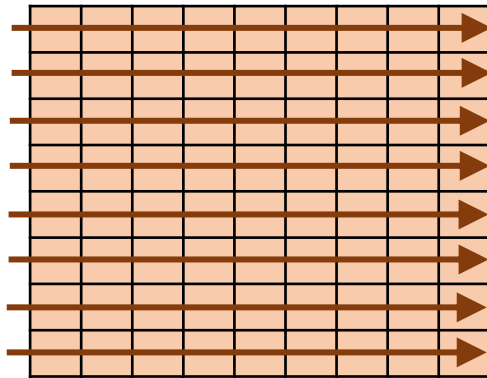


GPU Memory – Memory Coalescing

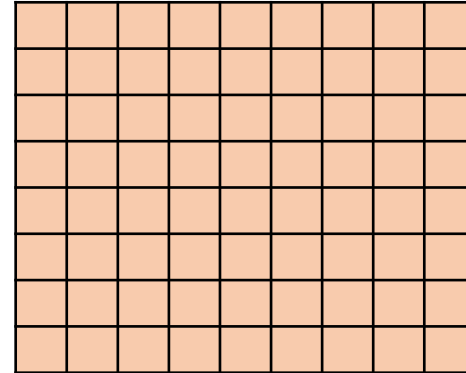
Within warp, we want threads to access the same memory block

- e.g., If we had the choice of parallelizing an array along rows vs. columns:

CPU



GPU

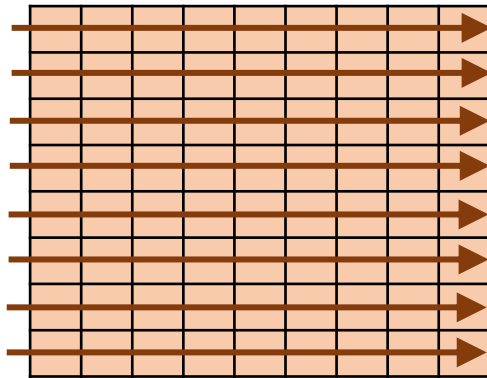


GPU Memory – Memory Coalescing

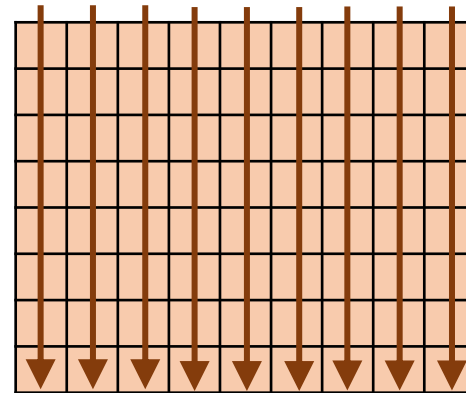
Within warp, we want threads to access the same memory block

- e.g., If we had the choice of parallelizing an array along rows vs. columns:
 - Different answer for CPU vs. GPU

CPU



GPU



GPU Coherence

GPU applications have minimal data sharing

- Traditional CPU protocols have too much overhead/complexity for GPUs
 - e.g., Invalidation traffic, sharer tracking, transient states
- Baseline GPU caches:
 - Write-through with coalescing
 - VI coherence: **V**: block is cached (R/W allowed), **I**: block is not cached (W allowed)
 - Supports release consistency

GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
```

directory	
flag	exp @ 45
x	exp @ 30
y	exp @ 15

memory (or L2)

```
// T2
*x = ...;
*y = ...;
fence;
*flag = 1;
```

GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
```

directory	
flag	exp @ 45
x	exp @ 30
y	exp @ 15

memory (or L2)	
----------------	--

writes
to mem

```
// T2
*x = ...;
*y = ...;
fence;
*flag = 1;
```



GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
```

directory	
flag	exp @ 45
x	exp @ 30
y	exp @ 15

memory (or L2)	
----------------	--

```
// T2
*x = ... ; expiry @ 30
*y = ... ;
fence;
*flag = 1;
```

GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
```

directory	
flag	exp @ 45
x	exp @ 30
y	exp @ 15

memory (or L2)	
----------------	--

```
// T2
*x = ...;
*y = ...;
fence;
*flag = 1;
```

writes
to mem



GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
```

directory	
flag	exp @ 45
x	exp @ 30
y	exp @ 15

memory (or L2)

```
// T2
*x = ...;
*y = ...; expiry @ 15
fence;
*flag = 1;
```



GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
```

directory	
flag	exp @ 45
x	exp @ 30
y	exp @ 15

memory (or L2)

```
// T2
*x = ...;
*y = ...;
fence; waits until max(30,15) = 30
*flag = 1;
```

GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
```

x and y self-invalidated

directory	
flag	exp @ 45
x	---
y	---

memory (or L2)

@ 30

```
// T2
*x = ...;
*y = ...;
fence; waits until max(30,15) = 30
*flag = 1;
```

GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
```

directory	
flag	exp @ 45
x	---
y	---

memory (or L2)

```
// T2
*x = ...;
*y = ...;
fence;
*flag = 1;
```

writes to mem

GPU Coherence

Temporal Coherence: Caches use time-based self-invalidations [HPCA 2013]

- Caches set/predict “expiry” timestamp when requesting blocks
 - Directory tracks max expiry timestamp of all sharers; no need to track sharer list
- On a write to X, directory responds with max sharer timestamp for X
 - Writes are performed immediately at the directory level (i.e., no store atomicity)
 - Fence/release forces thread to wait until max sharer timestamp (release consistency)

```
// T1
while(!*flag);
... = *x;
... = *y;
if flag==1, x and y are
guaranteed to be coherent
```

directory	
flag	exp @ 45
x	---
y	---

memory (or L2)

```
// T2
*x = ...;
*y = ...;
fence;
*flag = 1;
```

writes
to mem



GPU Coherence

QuickRelease: Improve temporal locality via write-back caches [HPCA 2014]

- On release, write back dirty data from prior stores
 - Use a store queue per cache to track order
- On acquire, self-invalidate (flush) entire cache
 - No complex directory needed
 - Assumes synchronization is rare

GPU Coherence

DeNovo: Improve temporal locality via selective self-invalidation [MICRO 2015]

- Dir tracks exclusive owners: writers of data, readers/writers of synch vars
 - Trades off directory complexity for better performance when synch is frequent
 - Supports data-race-free
- On synch, retain owned, touched and read-only data; flush everything else

// T1		// T2
A[] = ...;		
... = B[];		
lock(x);		...
unlock(x);	→	lock(x);
...		unlock(x);
		... = C[];

GPU Coherence

DeNovo: Improve temporal locality via selective self-invalidation [MICRO 2015]

- Dir tracks exclusive owners: writers of data, readers/writers of synch vars
 - Trades off directory complexity for better performance when synch is frequent
 - Supports data-race-free
- On synch, retain owned, touched and read-only data; flush everything else

retain A: dir can fwd
reqs to owner

// T1

A[] = ...;

... = B[];

lock(x);

unlock(x);

...

// T2

...

lock(x);

unlock(x);

... = C[];

GPU Coherence

DeNovo: Improve temporal locality via selective self-invalidation [MICRO 2015]

- Dir tracks exclusive owners: writers of data, readers/writers of synch vars
 - Trades off directory complexity for better performance when synch is frequent
 - Supports data-race-free
- On synch, retain owned, touched and read-only data; flush everything else

retain A: dir can fwd
reqs to owner

// T1

A[] = ...;

... = B[];

lock(x);

unlock(x);

...

// T2

...

lock(x);

unlock(x);

... = C[];

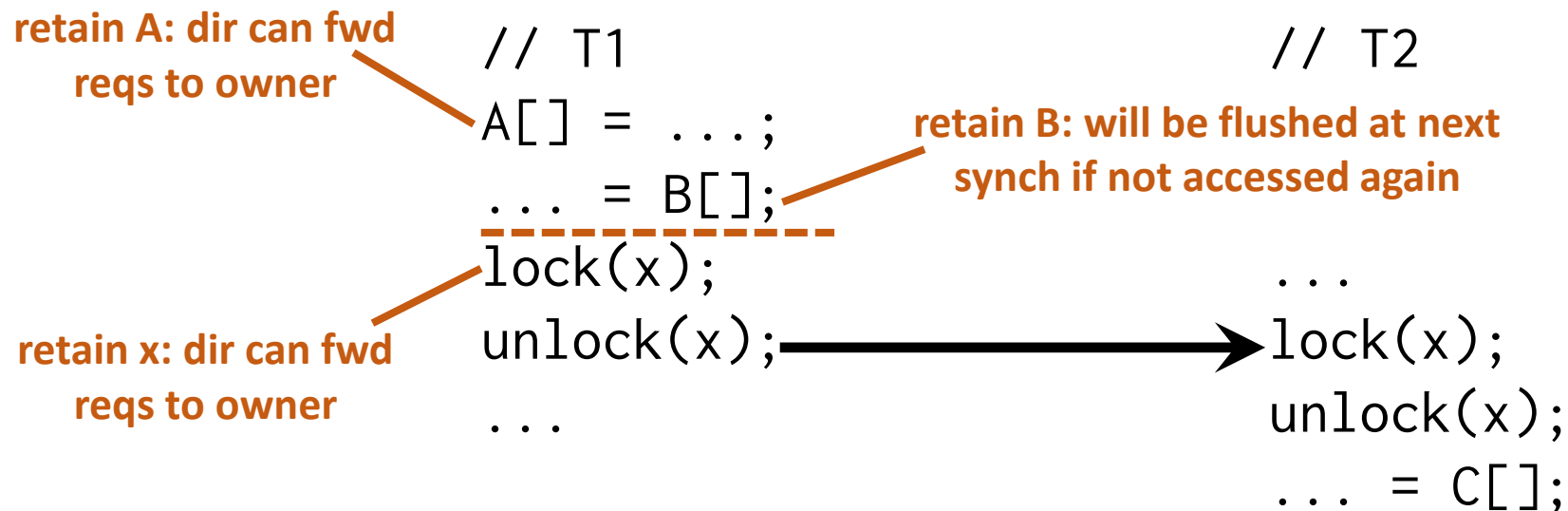
retain B: will be flushed at next
synch if not accessed again



GPU Coherence

DeNovo: Improve temporal locality via selective self-invalidation [MICRO 2015]

- Dir tracks exclusive owners: writers of data, readers/writers of synch vars
 - Trades off directory complexity for better performance when synch is frequent
 - Supports data-race-free
- On synch, retain owned, touched and read-only data; flush everything else



Heterogeneous-Race-Free

Extends data-race-free model to support **scopes** [ASPLOS 2014]:

- A **scope** is a subset of threads that may synchronize separately from the rest of the program
 - e.g., Threads in the same thread block may use barriers
- HRF guarantees sequential consistency if program is data-race-free in the presence of local synchronization (in addition to global synchronization)
 - Can eliminate the need for invalidations and flushes if synchronization is locally scoped; local cache will always have most up-to-date values

GPU Programming – CUDA

C-extension programming language

Extensions and API:

- Function types: `__global__`, `__device__`, `__host__`
- Variable types: `__shared__`, `__constant__`
- Build-in variables: `gridDim`, `blockDim`, `blockIdx`, `threadIdx`
- Memory: `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`, etc.
- Synchronization: `__syncthread()`, `atomicAdd()`, etc.

Program types:

- Device program (i.e., kernel): run on the GPU
- Host program: run on the CPU to call device programs

GPU Programming – CUDA

`__global__` function:

- Executed on device; only callable from host
- Defines a kernel function; must return void

`__device__` function:

- Executed on device; only callable from device
- Cannot have its address taken (i.e., cannot point to this function)

`__host__` function:

- Executed on host; only callable from host
- Can be used with `__device__` for common functions

GPU Programming – CUDA

`__global__` function:

- Executed on device; only callable from host
- Defines a kernel function; must return void

`__device__` function:

- Executed on device; only callable from device
- Cannot have its address taken (i.e., cannot point to this function)

`__host__` function:

- Executed on host; only callable from host
- Can be used with `__device__` for common functions

For functions executed on device:

- No recursion
- No static variable declarations inside the function
- No variable number of function arguments

GPU Programming – CUDA

e.g., Vector addition, kernel code:

```
// Pair-wise addition of vector elements
// One thread per addition

__global__ void
vectorAdd(float* iA, float* iB, float* oC)
{
    int idx = threadIdx.x + blockDim.x * blockIdx.x;
    oC[idx] = iA[idx] + iB[idx];
}
```

GPU Programming – CUDA

e.g., Vector addition, host code:

```
float* h_A = (float*) malloc(N * sizeof(float));
float* h_B = (float*) malloc(N * sizeof(float));
// ... initialize h_A and h_B

// allocate device memory
float* d_A, d_B, d_C;
cudaMalloc( (void**) &d_A, N * sizeof(float) );
cudaMalloc( (void**) &d_B, N * sizeof(float) );
cudaMalloc( (void**) &d_C, N * sizeof(float) );

// copy host memory to device
cudaMemcpy( d_A, h_A, N * sizeof(float),
            cudaMemcpyHostToDevice );
cudaMemcpy( d_B, h_B, N * sizeof(float),
            cudaMemcpyHostToDevice );

// execute the kernel on N/256 blocks of 256 threads each
vectorAdd<<< N/256, 256>>>( d_A, d_B, d_C);
```

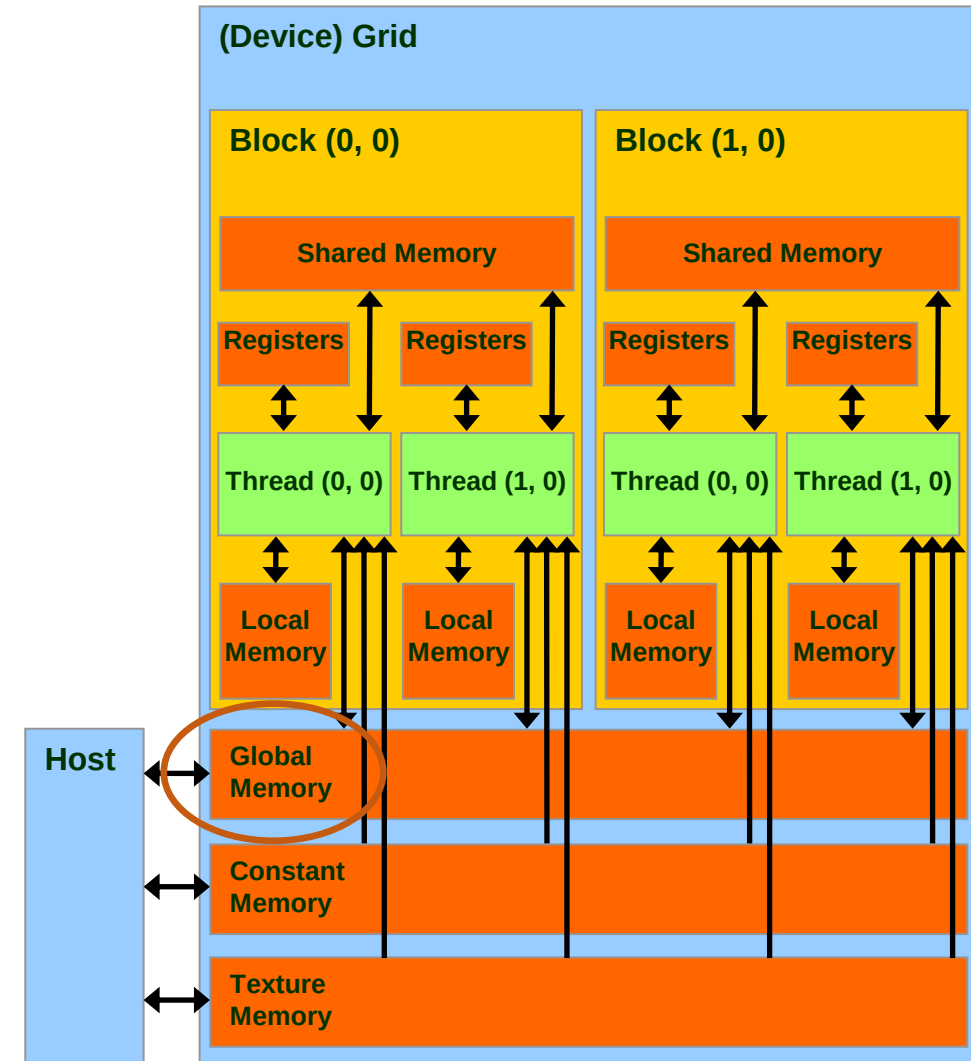
GPU Programming – CUDA

cudaMalloc():

- __host__ __device__
- Allocates object in device global memory
- Requires pointer and size parameters

cudaFree():

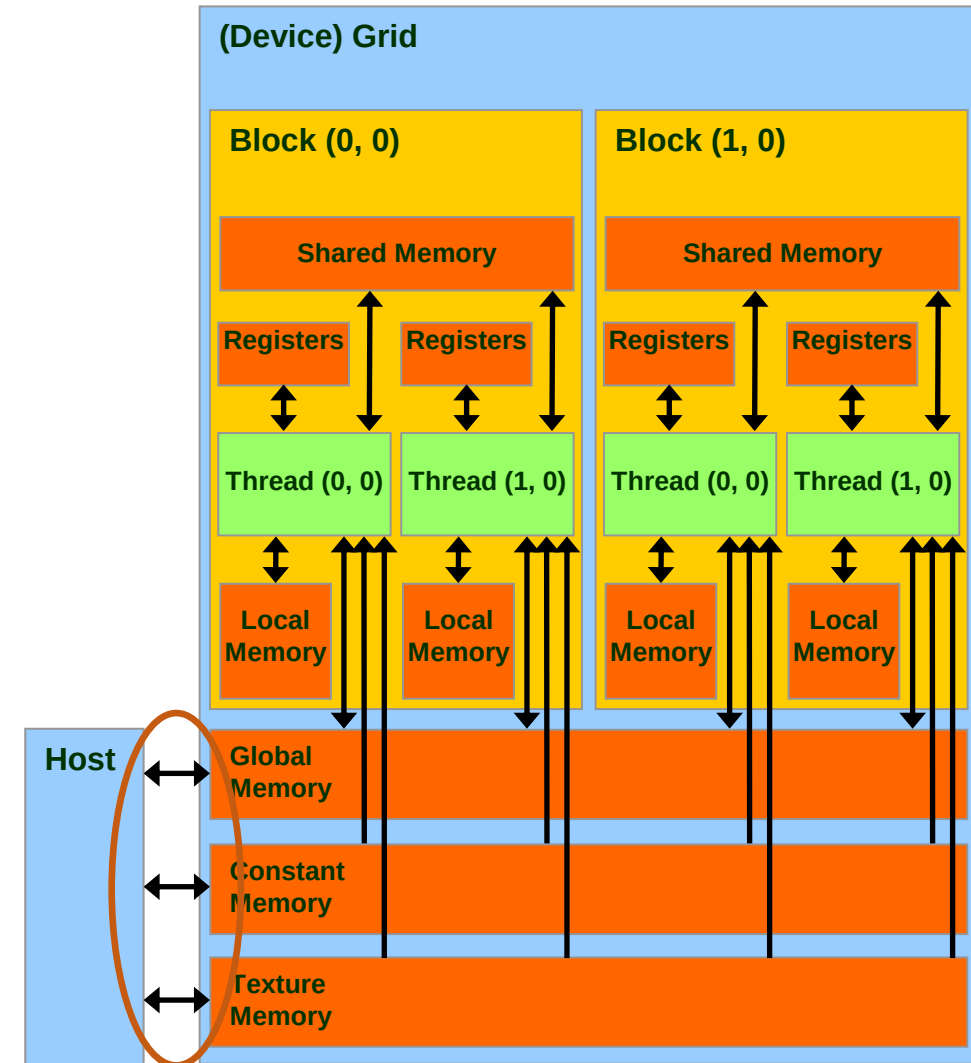
- __host__ __device__
- Frees object from device global memory
- Requires pointer parameter



GPU Programming – CUDA

`cudaMemcpy()`:

- `__host__`
- Memory data transfer
- Requires parameters: pointer to source, pointer to destination, size, type of transfer (host/device to host/device)



GPU Programming – CUDA

`__syncthreads()`:

- Synchronizes (i.e., barrier) all threads in a block
- Once all threads have reached this point, resume normal execution
- Allowed in conditional constructs, but only if the conditional is uniform across the entire thread block (i.e., either all threads see barrier or none do)

GPU Programming – CUDA

`__syncthreads()`:

- Synchronizes (i.e., barrier) all threads in a block
- Once all threads have reached this point, resume normal execution
- Allowed in conditional constructs, but only if the conditional is uniform across the entire thread block (i.e., either all threads see barrier or none do)

`__syncthreads()` only for threads in warp?

GPU Programming – CUDA

`__syncthreads()`:

- Synchronizes (i.e., barrier) all threads in a block
- Once all threads have reached this point, resume normal execution
- Allowed in conditional constructs, but only if the conditional is uniform across the entire thread block (i.e., either all threads see barrier or none do)

`__syncthreads()` only for threads in warp?

No need, always synchronized.

GPU Programming – CUDA

`__syncthreads()`:

- Synchronizes (i.e., barrier) all threads in a block
- Once all threads have reached this point, resume normal execution
- Allowed in conditional constructs, but only if the conditional is uniform across the entire thread block (i.e., either all threads see barrier or none do)

`__syncthreads()` only for threads in warp?

No need, always synchronized.

`__syncthreads()` for threads outside block?

GPU Programming – CUDA

`__syncthreads()`:

- Synchronizes (i.e., barrier) all threads in a block
- Once all threads have reached this point, resume normal execution
- Allowed in conditional constructs, but only if the conditional is uniform across the entire thread block (i.e., either all threads see barrier or none do)

__syncthreads() only for threads in warp?

No need, always synchronized.

__syncthreads() for threads outside block?

No semantics defined.

GPU Programming – CUDA

CUDA strengths:

- C extensions; small learning curve to program
- Free use of device memory; allows for reading and writing data at any location
- Overheads generally low

CUDA limitations:

- Some hardwired graphics capabilities remain hidden
- Need better tools for profiling, binary translation, etc.
- Difficult to find optimal values for CUDA parameters (e.g., number of threads per block, dimensions/shapes of blocks and grids, which on-chip memory resource to use)